

Report: USB Camera Face and Eye Detection Project

This report summarizes the troubleshooting, setup, and development work carried out in the conversation for a Python-based USB camera face and eye detection project.

Project objective

The goal is to use a USB camera connected to a Windows PC, process the live video feed with Python, detect human faces, draw a green rectangle around each detected face, detect eyes within the face region, and draw red rectangles around the detected eyes.

1. Initial technology choice

OpenCV was selected as the main library because it provides camera access, image processing, Haar Cascade classifiers, and drawing functions. The basic installation package is **opencv-python**.

The planned detection pipeline is:

Step	Action
1	Read a frame from the USB camera
2	Convert the frame to grayscale
3	Detect faces using a Haar Cascade
4	Draw a green rectangle around each face
5	Use the detected face as a region of interest
6	Detect eyes inside that region
7	Draw red rectangles around detected eyes

2. First OpenCV problem: CascadeClassifier missing

The first major error encountered was:

```
AttributeError: module 'cv2' has no attribute 'CascadeClassifier'
```

Possible causes considered included a conflicting file named cv2.py, an incorrectly installed package named cv2, a corrupted OpenCV installation, or Python importing a module other than the expected OpenCV package. The recommended package name is **opencv-python**, which provides the Python module named **cv2**.

Diagnostic commands discussed included:

```
import cv2
print(cv2.__version__)
print(cv2.__file__)
print(cv2.CascadeClassifier)
```

The purpose of printing cv2.__file__ was to determine exactly where Python was importing the cv2 module from.

3. OpenCV installation/build issue

A subsequent installation attempt failed with messages indicating that pip was trying to build OpenCV from source and failed while building a wheel. The visible error summary included:

```
ERROR: Failed building wheel for opencv-python
Failed to build opencv-python
error: failed-wheel-build-for-install
```

The troubleshooting approach was to avoid changing or removing the user's main Python 3.14 installation and instead isolate the camera project from the main Python environment.

4. Cleaning the Python environment

The user checked the installed packages in Python 3.14 and found NumPy present while OpenCV packages appeared to have been removed. NumPy was intentionally retained because it was not identified as the source of the issue.

Useful cleanup and verification commands discussed were:

```
py -3.14 -m pip uninstall opencv-python opencv-contrib-python opencv-python-headless
opencv-contrib-python-headless cv2 -y

py -3.14 -m pip cache purge

py -3.14 -c "import cv2; print(cv2.__file__)"
```

A ModuleNotFoundError after uninstalling OpenCV would be expected and would confirm that cv2 was no longer being imported from the environment.

5. Why a virtual environment was chosen

The user wanted to continue using Python 3.14 for other work. A virtual environment was therefore introduced to isolate project-specific packages from the main Python installation.

The important distinction established was:

Concept	Meaning
Python installation	The Python interpreter/version installed on the computer, such as Python 3.14.
Virtual environment (.venv)	An isolated environment for one project with its own installed packages.
Project files	The user's Python programs, stored outside the .venv folder.

It was also clarified that a virtual environment does not automatically provide an older Python version. The Python version used by the environment depends on the interpreter used to create it. For example:

```
py -3.14 -m venv .venv
```

creates a virtual environment based on Python 3.14.

6. VS Code project setup

A new project folder was opened in VS Code at:

```
D:\samples\imagedetection
```

The virtual environment was created inside the project folder:

```
py -3.14 -m venv .venv
```

The resulting structure was explained as:

```
imagedetection/
■
■■■■ .venv/ # Virtual environment and installed packages
■ ■■■ Include/
■ ■■■ Lib/
■ ■■■ Scripts/
■
■■■■ facialdetection.py # User's Python code
```

The user was specifically advised not to place their own Python programs inside .venv.

7. Activating and deactivating the environment

In the VS Code PowerShell terminal, the environment was activated using:

```
.\.venv\Scripts\Activate.ps1
```

The terminal then displayed the (.venv) prefix, confirming activation. The Python executable could be verified with:

```
python -c "import sys; print(sys.executable)"
```

The expected path should contain:

```
D:\samples\imagedetection\.venv\Scripts\python.exe
```

To leave the environment manually:

```
deactivate
```

It was also observed that VS Code automatically activated .venv when a new terminal was opened. This is normal when VS Code has detected or been configured to use the project's virtual environment.

8. GitHub Copilot clarification

The VS Code terminal displayed a message suggesting the use of Copilot CLI. It was clarified that GitHub Copilot is an optional AI coding assistant and is independent of the Python virtual environment. The project can use VS Code, GitHub, a virtual environment, and optionally Copilot without those components replacing one another.

9. Camera access testing

The initial camera program could not open the camera and produced a camera index error. Camera indexes 0, 1, and 2 were discussed, along with a test script to identify available cameras.

A useful test program was:

```
import cv2

for i in range(5):
    camera = cv2.VideoCapture(i)

    if camera.isOpened():
        print(f"Camera found at index {i}")
        camera.release()
    else:
        print(f"No camera at index {i}")
```

The user later successfully opened the camera, receiving:

```
Camera opened successfully  
Press Q to quit
```

However, the OpenCV display window remained black, indicating that the camera device could open but the captured frames were not displaying as expected.

10. DirectShow backend test

Because the project is running on Windows with a USB camera, the DirectShow backend was suggested:

```
camera = cv2.VideoCapture(0, cv2.CAP_DSHOW)
```

A more defensive camera test was:

```
import cv2  
  
camera = cv2.VideoCapture(0, cv2.CAP_DSHOW)  
  
if not camera.isOpened():  
    print("Error: Could not open the camera")  
    exit()  
  
print("Camera opened successfully")  
print("Press Q to quit")  
  
while True:  
    success, frame = camera.read()  
  
    if not success or frame is None:  
        print("Error: Could not read frame")  
        break  
  
    print("Frame size:", frame.shape)  
    cv2.imshow("USB Camera", frame)  
  
    if cv2.waitKey(1) & 0xFF == ord("q"):  
        break  
  
camera.release()  
cv2.destroyAllWindows()
```

The next troubleshooting step is to determine whether valid frames are being returned, try the appropriate camera index, close any other application using the camera, and compare the result with the Windows Camera application.

11. Intended final face and eye detection code

Once the cv2 installation and camera feed are functioning correctly, the project can use the following structure:

```
import cv2  
  
face_cascade = cv2.CascadeClassifier(  
    cv2.data.harcascades + "haarcascade_frontalface_default.xml"  
)  
  
eye_cascade = cv2.CascadeClassifier(  
    cv2.data.harcascades + "haarcascade_eye.xml"  
)
```

```

camera = cv2.VideoCapture(0, cv2.CAP_DSHOW)

if not camera.isOpened():
    print("Error: Could not open the camera")
    raise SystemExit

while True:
    success, frame = camera.read()

    if not success or frame is None:
        print("Error: Could not read frame")
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(50, 50)
    )

    for x, y, w, h in faces:
        # Green rectangle around the face
        cv2.rectangle(
            frame,
            (x, y),
            (x + w, y + h),
            (0, 255, 0),
            2
        )

        face_gray = gray[y:y + h, x:x + w]
        face_color = frame[y:y + h, x:x + w]

        eyes = eye_cascade.detectMultiScale(
            face_gray,
            scaleFactor=1.1,
            minNeighbors=10,
            minSize=(20, 20)
        )

        for ex, ey, ew, eh in eyes:
            # Red rectangle around each eye
            cv2.rectangle(
                face_color,
                (ex, ey),
                (ex + ew, ey + eh),
                (0, 0, 255),
                2
            )

    cv2.imshow("Face and Eye Detection", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

    camera.release()
    cv2.destroyAllWindows()

```

12. Current project status

The project has progressed through Python environment isolation, VS Code integration, virtual environment activation, camera index troubleshooting, and successful opening of a camera device. The remaining issues are:

Priority	Remaining task
1	Confirm that the installed cv2 package provides CascadeClassifier.
2	Confirm that the USB camera returns valid image frames rather than a black display.
3	Identify the correct camera index and backend if necessary.
4	Run the Haar Cascade face and eye detection program.
5	Tune detection parameters for lighting, distance, and camera quality.

Conclusion

The recommended architecture is to keep Python 3.14 installed for the user's general work, keep the face detection project isolated inside `D:\samples\imagedetection\.venv`, and install project-specific dependencies only inside that environment. The immediate technical focus should be on resolving the cv2 CascadeClassifier issue and verifying that the USB camera provides valid frames before enabling face and eye detection.